Course MA-IMF 1225 - "Lab Exploring HPC technologies"

Analysis of AMPM and IMP Prefetchers Compared with Next-line and Stride on an Out-of-Order Armv8 Architecture

Faruk Demirel
Jon Lumi
Computer Science Department
University of Bonn

July 05, 2026

Contents

List of Tables

List of Figures

Abstract

Hardware prefetching is an important technique for reducing the performance impact of long memory access latencies in modern processors. Here we evaluate several hardware data prefetchers for scientific and memory-intensive benchmark kernels on a simulated out-of-order Armv8 architecture using gem5. The main focus is on the Access Map Pattern Matching (AMPM) prefetcher and the Indirect Memory Prefetcher (IMP), which are compared against the more fundamental Next-line and Stride prefetchers. The evaluation considers prefetchers placed at both the L1 data cache and the shared L2 cache, using two DDR4 memory configurations. The studied benchmarks include STREAM Triad, sparse matrix-vector multiplication, matrix multiplication, merge sort, quick sort, and breadth-first search.

The report describes the simulated architecture, benchmark selection, prefetcher configurations, and evaluation metrics. The goal is to determine which memory-access patterns favor AMPM, IMP, Next-line, and Stride, and to explain the observed speedups using prefetch accuracy, coverage, late and unused prefetches, cache placement, and memory bandwidth.

1 Introduction

Memory performance is a major bottleneck in many high-performance computing (HPC) applications. While modern processors continue to increase core counts, instruction throughput, and vector capabilities, memory latency and bandwidth improve more slowly [1]. As a result, applications with large working sets or irregular memory accesses may spend a significant fraction of execution time waiting for data to arrive from the memory hierarchy. As such, hardware prefetching is one technique used to reduce this bottleneck [2]. A prefetcher tries to predict future memory accesses and fetch the corresponding cache lines before they are demanded by the processor. If the prediction is correct and timely, the demand access can hit in the cache instead of stalling on a memory access. However, prefetching can also be harmful if it fetches useless cache lines, consumes memory bandwidth, or evicts useful data from the cache [3].

This project studies hardware prefetching for a set of benchmarks using the gem5 simulator [4]. The focus is on two more advanced prefetching techniques: Access Map Pattern Matching (AMPM) [5] and the Indirect Memory Prefetcher (IMP) [6]. Then, these prefetchers are compared with two simpler baseline prefetchers: Next-line [7] and Stride [8].

The evaluation is performed on an out-of-order Armv8 architecture model. Prefetchers are evaluated at both the L1 data cache and L2 cache, and for two DDR4 memory configurations. The main goal is to analyze how different prefetchers behave across benchmarks with different memory access patterns, and determine when some prefetchers provide an advantage over others.

The rest of this report is organized as follows. Section 2 introduces background on memory prefetching and the evaluated prefetchers. Section 3 describes the simulation methodology, architecture model, benchmarks, prefetcher configurations, and metrics. Section 4 presents the evaluation results, including speedup, cache miss behavior, prefetch accuracy, coverage, and timeliness. Finally, Section 5 summarizes the main findings.

2 Background & Related Work

Different prefetchers target different types of memory access patterns. Simple spatial prefetchers, such as Next-line, assume that future accesses are close to the current access [7]. A Next-line prefetcher fetches the following cache line after an access, without requiring complex pattern detection. This can be effective for streaming or sequential access patterns, such as array traversal. On the other hand, Stride prefetchers detect constant-distance access patterns [8]. If a sequence of accesses follows a pattern such as consecutive cache lines or a fixed stride, the prefetcher can predict future addresses by adding the detected stride to the current address. Stride prefetching is especially relevant for scientific codes with regular array-based memory access patterns.

Access Map Pattern Matching (AMPM), as proposed by Ishii et al. [5], is a more sophisticated spatial prefetcher that extends the ideas behind simple Next-line and Stride prefetch-

ing. Instead of predicting only the next cache line or assuming a single constant stride, AMPM divides memory into regions and maintains an access map that records which cache lines within each region have recently been accessed, and by examining these access maps, the prefetcher searches for recurring spatial patterns and predicts future accesses based on previously observed relationships between cache lines. This enables AMPM to identify multiple access streams simultaneously, even when they exhibit different strides or are interleaved within the same memory region. Compared to Next-line prefetching, which only exploits immediate spatial locality, and Stride prefetching, which is limited to regular constant-distance accesses, AMPM can capture a much wider range of spatial access patterns while remaining effective for regular workloads. While this flexibility does add complexity given that the access maps must be continuously maintained and searched for matching patterns, it is comparable to that of the ALU, meaning it would not be too detrimental to modern processors [5].

The Indirect Memory Prefetcher (IMP), proposed by Yu et al. [6], targets memory access patterns that cannot be predicted from addresses alone. These indirect accesses, common in sparse matrix computations, graph traversal, and pointer-based data structures, occur when one memory address depends on data loaded by a previous operation (e.g., `A[B[i]]`). Unlike Next-line, Stride, and AMPM, which exploit spatial address patterns, IMP learns producer–consumer relationships between dependent loads to prefetch the final data before it is accessed. Concretely, IMP detects the sequentially streamed index array B and learns the address-generation rule $ADDR(A[B[i]]) = Coeff \times B[i] + BaseAddr$, where $Coeff$ is the element size of A and $BaseAddr$ its base address; since both are constant for a given pattern, index values streamed ahead can be used to compute the irregular data addresses before the program accesses them. In hardware, a prefetch table tracks the index streams and an indirect pattern detector learns $Coeff$ and $BaseAddr$ by correlating candidate index values with subsequent cache misses; restricting $Coeff$ to powers of two reduces the address computation to a shift and an addition [6].

Prior research has shown that no single hardware prefetcher configuration performs best in all situations [1]. For example, on systems with heterogeneous memory, prefetchers benefit from dynamically adapting their aggressiveness (degree and distance) to the bandwidth of the memory being accessed, prefetching aggressively while bandwidth is available and more conservatively as it saturates [9]. Studies on modern Arm-based HPC processors have demonstrated that prefetcher effectiveness depends strongly on workload memory access characteristics and available memory bandwidth, where in particular, applications with regular spatial access patterns benefit most from spatial prefetchers, while the optimal level of prefetch aggressiveness is workload-dependent [10]. Motivated by these findings, this project evaluates Next-line, Stride, AMPM, and IMP across a diverse benchmark suite with both regular and irregular memory access patterns. Beyond execution speedup, the evaluation considers prefetch accuracy, coverage, timeliness, unused prefetches, and cache miss behavior to provide a comprehensive assessment of prefetcher effectiveness.

Table 1: Simulated architecture parameters.

Component	Configuration
ISA	Armv8 (AArch64)
CPU model	gem5 `ArmO3CPU`, single core, 3 GHz
Pipeline width	4 (fetch/decode/rename/dispatch/issue/writeback/commit)
ROB size	128 entries
Physical integer registers	160
Physical FP/vector registers	160
Load/store queue	64 load entries + 64 store entries
Branch predictor	TournamentBP
Functional units	6 IntALU, 2 IntMultDiv, 4 FP ALU, 2 FP MultDiv, 4 SIMD
L1I cache	16 KiB, 8-way, 1-cycle latency
L1D cache	16 KiB, 8-way, 1-cycle latency, 16 MSHRs, 64 B lines
L2 cache	256 KiB shared, 16-way, 10-cycle latency, 20 MSHRs
Memory (1x)	DDR4-2400, 1 channel, 1 GiB
Memory (2x)	DDR4-2400, 2 channels, 1 GiB
Simulator	gem5, SE mode, `multisim` batch runs

3 Evaluation Methodology

All experiments use gem5 [4, 11] in syscall-emulation (SE) mode to simulate a single-core out-of-order Armv8 system. Every benchmark binary is instrumented with `m5` operations (`m5_reset_stats` / `m5_dump_stats`) around the computational kernel, so all reported statistics cover only the region of interest and exclude initialization and data-generation code. The experiments were organized with gem5's `multisim` framework, which allowed batches of configurations to be executed in parallel. In total, the final dataset contains more than 3,000 simulation runs: 12 no-prefetch baseline runs, 144 Next-line runs, 624 Stride runs, roughly 1,400 IMP runs collected over three tuning stages, and roughly 1,000 AMPM runs collected over two tuning stages.

3.1 Architecture Model

The simulated system uses gem5's `ArmO3CPU`, configured as a modern out-of-order processor, as summarized in Table 1. This aggressive out-of-order design is well suited for evaluating hardware prefetchers, as dynamic scheduling interleaves memory streams and exposes memory-level parallelism that prefetching can exploit.

The cache hierarchy consists of private L1 instruction and data caches and a shared L2 cache. The relatively small L1 data cache is intentionally chosen to ensure that benchmark working sets generate sufficient cache misses to stress the memory hierarchy. In each experiment, the evaluated prefetcher is attached either to the L1 data cache or the L2 cache, while the other cache level operates without a prefetcher.

Two DDR4-2400 memory configurations are evaluated: a single-channel configuration and a

dual-channel configuration with twice the available bandwidth. Since aggressive prefetching increases memory traffic, comparing these configurations reveals how available bandwidth influences prefetcher effectiveness.

3.2 Evaluated Prefetchers

Table 2: Next-line configuration parameters.

Parameter	Values
degree	{1, 2 (default), 4, 8, 16, 32}

The Next-line prefetcher (gem5 `TaggedPrefetcher`) represents a simple spatial prefetching strategy. It assumes that future memory accesses are likely to continue to the next cache line. This prefetcher has low complexity and can be effective for streaming memory accesses. For this prefetcher, the explored parameter and swept values can be found in Table 2.

Table 3: Stride configuration parameters.

Parameter	Values
degree	{1, 4 (default), 8, 16, 32}
distance	{0 (default), 1, 4, 8, 16, 32}

The Stride prefetcher (gem5 `StridePrefetcher`) detects regular per-PC address differences between memory accesses. Once a stable stride is detected, it issues prefetch requests for future addresses following the same stride. This prefetcher is expected to perform well for regular array-based access patterns. As for Stride, the exploration is a full cross product of degree and distance. Table 3.

Table 4: AMPM configuration parameters: Stage 1.

Parameter	Values
limit_stride	{0 (default), 2, 4, 8, 16, 32, 64}
start_degree	{1, 2, 4 (default), 8, 16, 32, 64}
hot_zone_size/access_map_table_entries (L2)	{64B/4096, 128B/2048, 256B/1024, 512B/512, 1KiB/256, 2KiB/128, 4KiB/64, 8KiB/32}
hot_zone_size/access_map_table_entries (L1D)	{64B/256, 128B/128, 256B/64, 512B/32, 1KiB/16, 2KiB/8, 4KiB/4, 8KiB/2}
access_map_table_assoc	{1, 2, 4, 8 (default), 16, 32, 64}
epoch_cycles	{64 000, 128 000, 256 000 (default), 512 000, 1 024 000}

AMPM tracks accesses within memory regions (hot zones) and attempts to identify spatial patterns from the observed access map. AMPM was tuned in two stages. Stage 1 (Table 4) swept one parameter at a time around the gem5 default. Because the hot-zone size and the number of access-map-table entries jointly determine the memory range covered by the access maps, these two parameters were varied together (entries scaled inversely to the zone size), with separate organizations for L1D and L2 placement so as to fit in each of them, as

Table 5: AMPM configuration parameters: Stage 2.

Parameter	**Values**
limit_stride	{8, 16} (L1D); {8} (L2)
start_degree	{8, 16}
hot_zone_size/access_map_table_entries (L2)	{512B/512, 1KiB/256}
hot_zone_size/access_map_table_entries (L1D)	{1KiB/16}
access_map_table_assoc	{4} (L1D); {4, 16} (L2)
epoch_cycles	{128 000, 1 024 000}

also done in [5]. Stage 2 (Table 5) combined the strongest Stage-1 values into cross-product configurations.

Table 6: IMP configuration parameters: Stage 1 (one parameter varied at a time).

Parameter	**Values**
max_prefetch_distance	{4, 8, 16 (default), 32, 64}
prefetch_threshold	{1, 2 (default), 3, 4}
streaming_distance	{2, 4 (default), 8, 16}
stream_counter_threshold	{2, 4 (default), 8}
pt_table_entries (prefetch table)	{8, 16 (default), 32, 64}
ipd_table_entries (indirect pattern detector)	{2, 4 (default), 8, 16}
addr_array_len (tracked miss addresses)	{2, 4 (default), 8, 16}

Table 7: IMP configuration parameters: Stages 2 and 3 (combined configurations).

Parameter	**Values**
streaming_distance (Stage 2)	{2, 8, 16}
streaming_distance (Stage 3)	{32, 64}
combined with:	
stream_counter_threshold	{2, 8}
max_prefetch_distance	{8, 32}
pt_table_entries	{8, 32}
ipd_table_entries	{8, 16}
addr_array_len	{8, 16}

IMP targets indirect memory access patterns, where one loaded value is used to compute the address of a later memory access. The gem5 implementation pairs a stream detector with an indirect pattern detector (IPD) that learns `A[B[i]]`-style relations between an index load and the resulting indirect access; because the prefetcher must inspect the accessed data to learn these index-to-address relations, it is notified only on demand read misses in all runs (not on writes, instruction fetches, or prefetch hits). IMP was tuned in three stages. Stage 1 (Table 6) swept each parameter individually around the gem5 default; the table associativities were adjusted together with the table capacities.

Table 8: Benchmark selection and problem sizes.

Benchmark	Problem size	Expected memory behavior
STREAM Triad [14]	10^6 doubles per array (3 arrays, ∼24 MB)	Regular streaming, bandwidth-bound
SpMV (CSR)	8192×65536, 524,155 non-zeros	Streaming CSR arrays + indirect `vec[cols[j]]`
Matrix mult.	128×128, blocked	Regular accesses with strong cache reuse
Merge sort	131,072 random integers	Sequential merging of sub-arrays
Quick sort	131,072 random integers	Partition scans, data-dependent branches
BFS (CSR graph)	173,984 nodes, 3,939,240 edges	Sequential edge scans + indirect visited accesses

Stages 2 and 3 (Table 7) combined the per-parameter winners into multi-parameter configurations built around the streaming distance, which Stage 1 identified as the most influential parameter. Stage 2 explored 27 combinations at streaming distances $\{2, 8, 16\}$, including deliberately aggressive (e.g., `aggr_s16_sth2_d32_ipd16_addr16`) and conservative (e.g., `cons_s2_sth8_pt8`) anchor configurations. Since the most aggressive streaming settings kept winning, Stage 3 re-ran the 13 best combination templates at streaming distances $\{32, 64\}$ (26 configurations, e.g., `s32_sth2_dist32_ipd16_addr16`), giving 75 distinct IMP configurations in total.

For each prefetcher, both default and tuned configurations are considered. The tuned configurations vary relevant prefetcher parameters such as prefetch degree, distance, thresholds, table sizes, and other prefetcher-specific parameters. The final comparison reports both default behavior and the best-performing tuned configuration.

3.3 Benchmarks

The evaluation uses six benchmark kernels with different memory access characteristics, displayed in Table 8. These benchmarks cover regular streaming, dense compute with cache reuse, sparse indirect computation, sorting, and graph traversal, so the prefetchers are evaluated across a range of access patterns. The kernels are adapted from the microbenchmark set of Nowatzki's computer-architecture course [12] and, for BFS, from Lowe-Power's gem5 course assignments [13]; STREAM Triad is derived from McCalpin's STREAM benchmark [14].

Although BFS is algorithmically irregular, the implementation used in this project stores graph data in a compact adjacency-array (CSR) format. Therefore, part of its execution scans the edge array sequentially. This means that BFS can expose both irregular accesses and spatially predictable memory streams, which is relevant when interpreting the performance of IMP compared with Next-line and Stride. Similarly, SpMV mixes fully regular streaming over the `val`, `cols`, and `rowDelim` arrays with the indirect gather `vec[cols[j]]`.

The no-prefetch baseline characterization already separates the benchmarks into two groups. STREAM Triad, SpMV, and BFS are strongly memory-bound, with baseline IPCs of only 0.40, 0.36, and 0.37 and mean load-to-use latencies of 208, 131, and 88 cycles, respectively. Matrix multiplication (IPC 1.93), merge sort (IPC 1.09), and quick sort (IPC 0.84) are much less memory-latency-bound, with mean load-to-use latencies below 30 cycles. This bound on the achievable benefit is important context for all later results.

Table 9: Evaluation metrics.

Metric	Description
Simulation time	gem5 `simSeconds` for the benchmark region of interest
Speedup	No-prefetch time divided by prefetcher time (same benchmark/memory)
IPC	Instructions per cycle in the region of interest
Demand misses / miss rate	Demand misses (absolute, and divided by demand accesses) at L1D and L2
Prefetches issued	Prefetch requests sent to memory by the installed prefetcher
Useful prefetches	Prefetched lines later referenced by a demand access
Unused prefetches	Prefetched lines evicted without being referenced
Prefetch accuracy	useful / issued
Prefetch coverage	useful / (useful + demand misses at the installed level)
Late prefetches	Prefetches still in flight when the demand access arrives
Late ratio	late / issued
Unused ratio	unused / issued
Prefetcher read traffic	Bytes and bytes/s read from memory by the prefetcher
DRAM bandwidth utilization	Total DRAM bandwidth divided by peak bandwidth
Load-to-use latency	Mean cycles from load issue to data use, measured at the core

3.4 Experimental Design

Each benchmark is simulated under multiple prefetcher configurations. The experiments vary the prefetcher family, prefetcher configuration, cache placement, and memory configuration. A no-prefetch baseline is simulated for each benchmark and memory configuration and is the reference point for all speedups.

With that in mind, there are 24 evaluation cases (6 benchmarks $\times$ 2 placements $\times$ 2 memories) per family. For each case, the analysis reports both the *default* configuration of each family and the *best tuned* configuration (highest speedup). Comparing the two separates the benefit of the prefetching technique itself from the benefit of parameter tuning. Because plotting every configuration would be unreadable, the metric analysis additionally uses a selection methodology: per benchmark, case, and family, it selects the default configuration, the top-3 configurations by speedup, top-3 by coverage, top-3 by accuracy, lowest-3 by unused-prefetch ratio, and lowest-3 by late-prefetch ratio.

3.5 Metrics

Several metric groups are used to evaluate prefetcher behavior, summarized in Table 9. Speedup measures the overall performance improvement relative to the no-prefetch baseline. Prefetch quality metrics (accuracy, coverage, unused ratio, late ratio) explain whether a prefetcher issues useful, timely requests. Cache metrics show whether demand misses are actually removed. Memory-pressure metrics (prefetcher read traffic, DRAM bandwidth utilization, queue latencies) show whether a prefetcher helps by hiding latency or hurts by overloading the memory system. The mean load-to-use latency connects memory behavior back to what the core experiences.

The main speedup metric is calculated relative to the no-prefetch baseline:

$$\text{Speedup} = \frac{T_{\text{no-prefetch}}}{T_{\text{prefetcher}}} \tag{1}$$

where $T_{\text{no-prefetch}}$ is the simulated time without hardware prefetching and $T_{\text{prefetcher}}$ is the simulated time with a given prefetcher configuration, always for the same benchmark and memory system. Prefetch accuracy and coverage are calculated as:

$$\text{Accuracy} = \frac{\text{Useful prefetches}}{\text{Issued prefetches}} \qquad \text{Coverage} = \frac{\text{Useful prefetches}}{\text{Useful prefetches} + \text{Demand misses}} \tag{2}$$

where the demand misses are counted at the cache level where the prefetcher is installed. For accuracy and coverage, higher is better; for unused ratio, late ratio, miss rates, prefetcher read traffic, DRAM queue latency, and load-to-use latency, lower is better. All plots and ranked tables state the ranking direction explicitly.

These metrics must be analyzed together because high speedup does not necessarily imply high accuracy. A prefetcher may improve performance despite low measured accuracy if it generates enough timely or overlapping requests and the memory system has spare bandwidth. Conversely, a prefetcher may have high accuracy but low speedup if its prefetches arrive too late, cover too few misses, or the benchmark is not memory-bound. Several concrete examples of both effects appear in the results.

4 Evaluation Results

4.1 Overall Speedup Comparison

Figure 1 shows, for each of the four placement/memory cases, the best-performing configuration of every prefetcher family on every benchmark, normalized to the no-prefetch baseline (1.0). The red line on each bar marks the default configuration of that family, so the gap between the line and the bar top is the tuning gain. Table 10 summarizes the winning family per case.

Table 10: Winning prefetcher family (best tuned configuration) and speedup per evaluation case.

Benchmark	**L1D, DDR4 1x**	**L1D, DDR4 2x**	**L2, DDR4 1x**	**L2, DDR4 2x**
STREAM Triad	Stride (1.18)	AMPM (1.47)	Stride (1.23)	Next-line (1.69)
SpMV	IMP (1.78)	IMP (1.69)	IMP (1.77)	IMP (1.71)
BFS	Stride (1.34)	Stride (1.35)	Stride (1.29)	Stride (1.31)
Merge sort	AMPM (1.19)	AMPM (1.20)	AMPM (1.16)	AMPM (1.17)
Quick sort	AMPM (1.18)	Next-line (1.18)	Next-line (1.14)	Next-line (1.13)
Matrix mult.	Stride (1.07)	Stride (1.08)	Next-line (1.10)	Next-line (1.12)

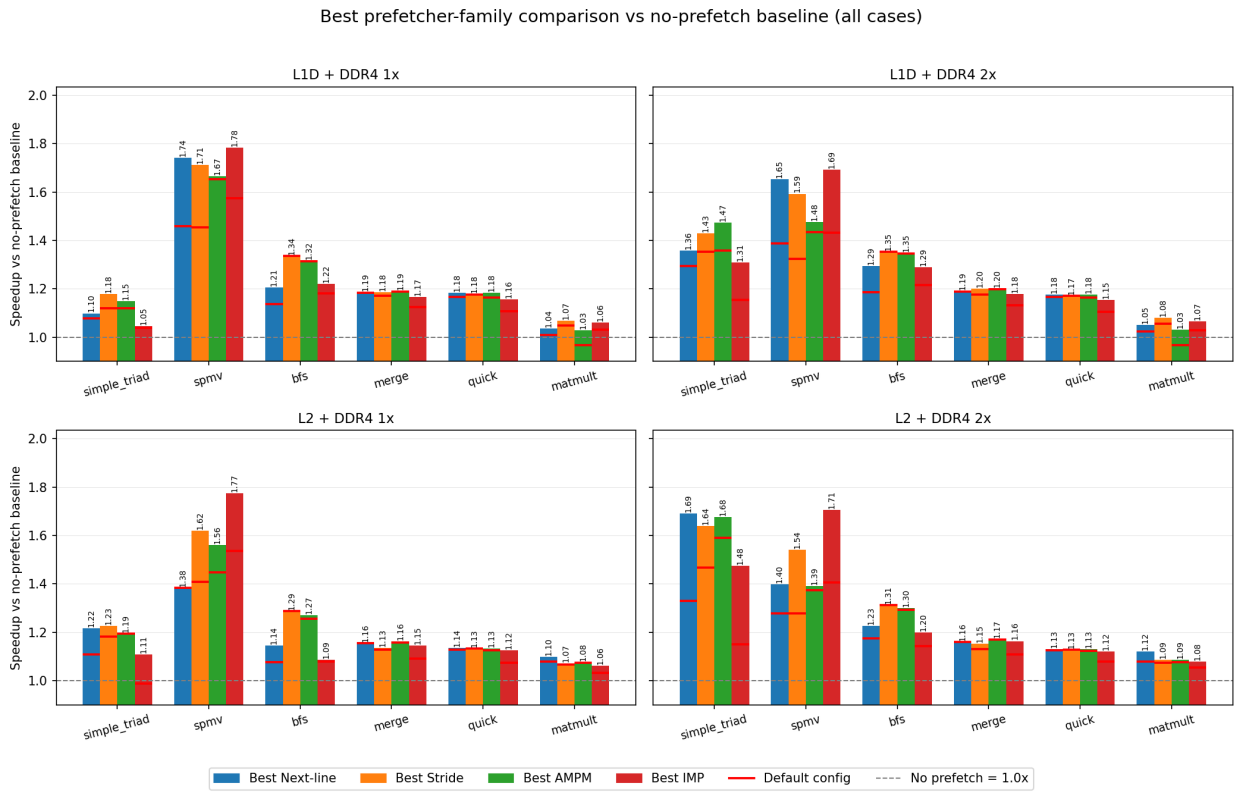

Figure 1: Best configuration per prefetcher family vs. the no-prefetch baseline, for all benchmarks and all four placement/memory cases. Red lines mark the default configuration of each family.

Three main observations follow from the overall comparison.

First, *no single prefetcher wins everywhere.* In these experiments, the winning prefetcher is strongly associated with the benchmark's dominant observed access pattern. Across the 24 evaluation cases, Stride wins 8 (all four BFS cases plus part of Triad and matmult), AMPM wins 6 (all four merge-sort cases plus two more), Next-line wins 6, and IMP wins 4, but IMP's four wins are all four SpMV cases, the only benchmark whose miss stream is dominated by a true indirect pattern. This matches the design intent of each prefetcher.

Second, *the differences between families are often small, except where the specialized pattern exists.* On merge sort, quick sort, and matmult, the four families' best configurations lie within a few percent of each other (e.g., quick sort at L1D/DDR4 1x: AMPM 1.183, Next-line 1.183, Stride 1.178, IMP 1.156). In contrast, on SpMV at L2/DDR4 1x, IMP (1.77) beats the best Next-line (1.38) by almost 30 percentage points, and on BFS at L2 IMP (1.09) loses to Stride (1.29) by a similar margin. In this study, the added complexity of AMPM and IMP pays off most clearly when their targeted access pattern is prominent.

Third, *the magnitude of the achievable speedup tracks how memory-bound the baseline is.* The memory-bound benchmarks SpMV (up to 1.78), Triad (up to 1.69), and BFS (up to 1.35) show the largest gains, while matmult, with a baseline IPC of 1.93 and strong cache

Table 11: Tuning gain: geometric mean of (best tuned speedup / default speedup) over the four placement/memory cases of each benchmark.

Benchmark	Next-line	Stride	AMPM	IMP
STREAM Triad	1.105	1.065	1.041	1.132
SpMV	1.117	1.183	1.031	1.170
BFS	1.064	1.000	1.006	1.037
Merge sort	1.002	1.014	1.003	1.045
Quick sort	1.009	1.000	1.010	1.043
Matrix mult.	1.027	1.014	1.034	1.028

reuse, is limited to 1.03–1.12 regardless of prefetcher. Over all 24 cases, the geometric-mean speedups of the best-tuned configurations are remarkably close: Stride 1.27, AMPM 1.25, IMP 1.24, and Next-line 1.24. For the geometric mean over the 24 evaluated cases, tuned Stride is competitive with AMPM and IMP. This suggests that, for this benchmark mix, complex prefetchers provide their clearest benefit on specific matching patterns rather than uniformly across all workloads.

4.2 Default Versus Tuned Configurations

The red default-configuration markers in Figure 1 and the per-benchmark figures in Section 4.8 allow the tuning benefit to be separated from the technique itself. Table 11 reports the geometric-mean ratio between the best-tuned and the default speedup over the four cases of each benchmark.

Tuning matters most exactly where prefetching matters most. On the memory-bound benchmarks, tuning adds up to 18% additional speedup (Stride on SpMV: default 1.37 vs. tuned 1.62 geometric mean; IMP on SpMV: 1.49 vs. 1.74; IMP on Triad: +13%). The dominant tuned parameter for the simple prefetchers is aggressiveness: the best Triad and SpMV configurations use high degrees/distances (e.g., `deg16`–`deg32` Next-line, `deg32_dist1` Stride), which keep many more memory requests in flight. For IMP, the multi-parameter Stage-2/3 configurations (e.g., `s32_sth2_dist32_ipd16_addr16`: larger streaming distance, lower stream threshold, longer prefetch distance, larger pattern tables) consistently beat every single-parameter Stage-1 variant on SpMV, confirming that IMP's parameters interact and single-parameter tuning is insufficient.

On the sorting benchmarks and BFS, defaults are already close to optimal. Stride gains nothing from tuning on BFS and quick sort (its default wins BFS outright), and AMPM gains under 1% on BFS, merge, and quick. Interestingly, IMP has the *largest average* tuning gain (geometric mean 1.152 default vs. 1.238 tuned over all cases) and the *worst default* behavior of all families; its default configuration is also the only one that produces a slowdown on Triad at L2/DDR4 1x (0.99). AMPM's default is likewise slightly harmful on matmult at L1D (0.97). With tuning, no family loses to the no-prefetch baseline in any of the 24 cases.

4.3 Effect of Cache Placement

Comparing L1D against L2 placement for the same benchmark and family shows a consistent, benchmark-dependent split.

L1D placement wins for latency-sensitive, irregular benchmarks. On BFS, merge, quick, and SpMV, the best L1D configurations beat the best L2 configurations for almost every family, typically by 2–10%. The largest placement effect in the whole study is Next-line on SpMV, which achieves a 1.70 average best speedup at L1D but only 1.39 at L2 (ratio 1.22): at L1D, prefetched lines land next to the core and also cover L1D misses that would otherwise still pay the L1D→L2 round trip. The results do not show L1D pollution as the dominant limiter for these tuned configurations; either unused ratios remain low, or the displaced lines appear to have limited reuse.

L2 placement wins for streaming and reuse-heavy benchmarks. On Triad, every family is clearly better at L2 (L1D/L2 speedup ratio 0.85–0.91), and on matmult L2 placement is also preferred (0.94–1.00). For Triad, streams touch each line exactly once, so there is no reuse for L1D to protect; prefetching into the larger L2 with 20 MSHRs sustains more outstanding memory requests, which is what a bandwidth-bound stream needs. The clearest evidence is coverage, where on matmult, the same prefetcher families reach 70–86% coverage at L2 but essentially 0% at L1D. One likely explanation is that, at L1D, prefetched lines either do not remain resident long enough to be counted as useful or are constrained by request pressure, whereas the larger L2 can retain them longer.

In these results, placement is more strongly influenced by benchmark behavior than by prefetcher family: irregular/latency-bound codes prefer L1D placement for timeliness, while streaming/bandwidth-bound codes prefer the L2's capacity and request concurrency.

4.4 Effect of Memory Bandwidth

Doubling memory channels seems to change prefetcher effectiveness in three distinct ways.

With increased bandwidth, prefetching benefits grow strongly. On Triad, the best speedup of every family is 1.28–1.34× higher under DDR4 2x than under 1x (e.g., Next-line at L2 goes from 1.22 to 1.69). Under DDR4 1x the baseline already utilizes about 65% of DRAM bandwidth and prefetch configurations push it further toward saturation, so prefetching mainly reorders traffic; the average DRAM queue latency stays in the tens of thousands of ticks. Under 2x, the same prefetch streams have headroom, queue latency roughly halves, and aggressive prefetching converts directly into performance. This behavior is consistent with the expected interaction between prefetch aggressiveness and available memory bandwidth [9].

Prefetching benefit shrinks slightly when workload is more latency-bound. On SpMV the relative benefit of prefetching is lower under 2x (ratios 0.89–0.98), since the second channel already reduces the baseline's effective latency (baseline time drops from 10.6 ms to 9.0 ms), so there is less stall time left for the prefetcher to hide. BFS sits in between (+2–8%). This shows that "more bandwidth helps prefetching" is only true when bandwidth, not latency,

is the binding constraint.

Compute-bound workloads are insensitive to higher memory bandwidth. Merge, quick, and matmult change by less than 2% between memory systems, confirming they do not stress DRAM bandwidth (DRAM utilization is 1–4% for the sorts and matmult).

4.5 Cache Miss Rate Analysis

At the installed level, effective prefetchers strongly reduce demand misses. On quick sort at L1D, all spatial prefetchers reach coverage above 0.85 and drive the L1D miss rate from 2.7% (baseline region average) down to 0.1–0.2%; on merge at L1D the best configurations reach 2–5% L1D miss rate. On SpMV at L2, IMP's indirect prefetches cut the L2 miss rate to about 0.20 while covering 38% of would-be misses. On BFS, AMPM at L1D reduces the L2 miss rate to 0.28–0.29 versus roughly 0.42 for Stride, but Stride is still faster, because Stride's L1D-level timeliness matters more than AMPM's extra L2-level coverage.

An interesting observation though, is that prefetchers nearly eliminate L1D demand misses, yet the speedup saturates at about 1.18. The reason is visible in the baseline characterization: quick sort's mean load-to-use latency is only 4.7 cycles, so memory stalls are a minor part of its execution time; the remainder is branch- and dependency-limited sorting work that no prefetcher can accelerate.

Another is Triad at L1D, where the measured coverage and accuracy are near zero (below 1–2%), yet speedups of 1.18–1.47 are observed. Here almost every prefetched line is either still in flight when the demand arrives (counted late, not useful) or the demand merges with the in-flight prefetch in the MSHRs. The lines rarely become clean "useful prefetch" hits in the small L1D, but the overlap still hides a large part of the DRAM latency. Miss counters alone would suggest these prefetchers do nothing; the speedup and the DRAM-level statistics show otherwise. This is a strong argument for evaluating prefetchers with timeliness and memory-pressure metrics, not only hit/miss statistics.

4.6 Prefetch Accuracy and Coverage

Figure 2 combines the four diagnostic prefetch-quality metrics into one matrix, showing for each metric only the benchmark and placement/memory case where it is most informative: accuracy for SpMV (a), coverage for quick sort (b), unused-prefetch ratio for BFS (c), and late-prefetch ratio for STREAM Triad (d). Each point is one selected configuration; color encodes the family and the marker shape encodes why the configuration was selected (default, top-3 speedup, top-3 coverage, top-3 accuracy, lowest-3 unused, lowest-3 late). The full per-benchmark versions of these plots, covering all four cases per benchmark and metric, are available as supporting material together with the ranked CSV tables. This subsection discusses panels (a) and (b); Section 4.7 discusses panels (c) and (d).

The SpMV panel (a) shows the clearest separation between prefetching philosophies in the whole study. IMP's best configurations combine accuracy around 0.55–0.58 with the highest speedups (right side of the panel), because the indirect pattern detector only issues a

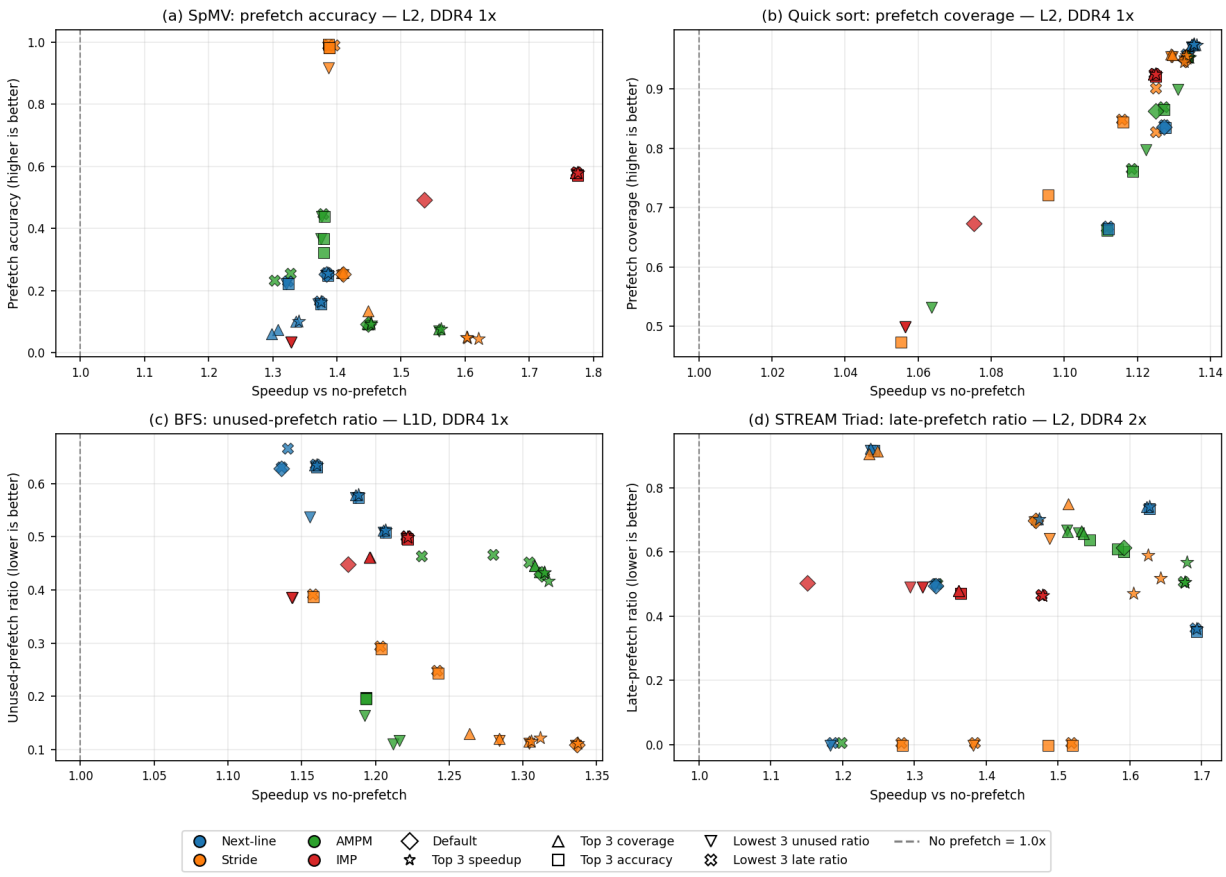

Figure 2: Diagnostic prefetch-quality metrics vs. speedup for selected configurations. Each panel shows the metric most relevant for one benchmark, in its most representative placement/memory case: (a) SpMV accuracy, (b) quick-sort coverage, (c) BFS unused-prefetch ratio, (d) STREAM Triad late-prefetch ratio. Higher is better for accuracy and coverage; lower is better for unused and late ratios.

prefetch once a `vec[cols[j]]` relation is confirmed. The spatial prefetchers reach comparable speedups through much more aggressive streaming behavior, with substantially lower measured accuracy: the best Stride configuration at L2 reaches 1.62 with an accuracy of just 0.04 by streaming deep through the regular CSR arrays. Notably, the *highest-accuracy* points in the panel (orange squares near accuracy 1.0) are conservative Stride configurations with speedups of only ∼1.39: they waste nothing but also cover too little. Accuracy alone identifies neither the best nor the worst configuration.

The quick-sort panel (b) shows the opposite regime: coverage is uniformly high (0.5–0.97) and the speedup range is compressed (1.05–1.14), so all points cluster. When a benchmark's misses are easy to cover but memory stalls are small, prefetch quality metrics stop discriminating — the bottleneck has moved elsewhere. IMP at L2 on quick sort is the extreme illustration: 0.93 accuracy, 0.92 coverage, only 7% late, the best quality profile of any configuration in the study — and still the *lowest* speedup of the four families (1.12). Excellent prefetching of a non-bottleneck is worth little.

4.7 Timely, Late, and Unused Prefetches

Panels (c) and (d) of Figure 2 examine the two failure modes of prefetching: fetching lines that are never used, and fetching lines too late.

On BFS (panel c), the family ranking is strongly consistent with the unused-prefetch behavior. Stride's default configuration wins every BFS case while wasting only 7–11% of its prefetches: its per-PC stride detection appears to focus selectively on the sequential edge-array scan and avoids much of the useless traffic generated by the other prefetchers. Next-line, IMP, and AMPM all waste 40–51% of their prefetches at L1D — roughly half of their memory traffic is useless, which both pollutes the 16 KiB L1D and consumes bandwidth (Next-line `deg32` raises DRAM utilization to 0.49 versus Stride's 0.27, for a *lower* speedup). BFS demonstrates that on mixed regular/irregular patterns, selectivity beats aggressiveness.

On Triad (panel d), late ratio is the most informative metric. Many selected Triad configurations, especially at L1D and also several at L2, have substantial late ratios of roughly 0.3–0.8. This is expected for a streaming kernel whose access stream can expose long DRAM latency faster than some prefetch configurations can fully hide it. The key observation is that late prefetches are not worthless here: configurations with substantial late ratios still deliver high speedups, because a late prefetch may still overlap part of the DRAM access with computation on earlier elements. Therefore, lateness reduces the potential benefit of a prefetch, but it does not always eliminate it. Late prefetches are most harmful when they also increase bandwidth pressure, queue occupancy, or cache contention. Under DDR4 2x, the extra bandwidth reduces this penalty, so the largest Triad speedups at L2 (1.64–1.69) can still coexist with moderate late ratios around 0.35–0.55. In contrast, on latency-bound SpMV, IMP's tuned configurations lower the late ratio from 0.48 for the default configuration toward 0.19–0.40 while increasing distance, suggesting that improved timeliness is an important part of IMP's tuning gain.

Overall, unused prefetches are the dominant failure mode for irregular benchmarks (BFS, SpMV at L1D), while late prefetches are the dominant, but partially harmless, mode for streaming benchmarks (Triad, matmult at L2, where late ratios exceed 0.9).

4.8 Per-Benchmark Discussion

Here each benchmark is summarized individually, referring back to the per-case panels of Figure 1.

Triad is the pure bandwidth-bound stream (baseline IPC 0.40, DRAM utilization $\sim$0.65 under 1x). Under DDR4 1x, all families are capped at 1.05–1.23 because the memory system is already near saturation; the ranking barely matters. Under DDR4 2x the picture changes completely: at L2, Next-line `deg8` reaches 1.69, AMPM 1.68, and Stride 1.64, while IMP trails at 1.48, its stream detector is a functional but weaker streaming prefetcher, and its indirect machinery has nothing to detect. The simple spatial prefetchers thus win the simplest pattern, and the benefit of prefetching here is almost entirely a function of available bandwidth and placement (L2 > L1D by 9–15%).

SpMV is IMP's showcase and the only benchmark it sweeps: 1.78/1.69/1.77/1.71 across the four cases. The mechanism differs by placement. At L2, IMP wins on quality: 0.55–0.58 accuracy from genuine `vec[cols[j]]` pattern detection, versus 0.04–0.25 for the spatial prefetchers, with far fewer late prefetches (0.38 vs. 0.96 for Stride). At L1D, all families converge to 1.6–1.8 by different means, where Next-line and IMP tolerate high unused ratios (0.74–0.78) to cover the streaming CSR arrays, while Stride stays precise but late. That even Next-line reaches 1.74 at L1D shows how much of SpMV's miss stream is the regular part of the kernel. IMP's edge comes from additionally covering part of the irregular gather. This is also the benchmark where parameter tuning pays the most for every family, seen as the default markers in Figure 1 sit 12–18% below the tuned bars.

Matmult is the least memory-bound benchmark (baseline IPC 1.93; the blocked 128×128 working set largely fits in L2), and prefetching yields only 1.03–1.12. L2 placement is more effective for obtaining these small gains, where at L2 the prefetchers achieve 70–86% coverage of the (few) L2 misses, and on the other hand at L1D coverage is essentially zero and AMPM's default even causes a 3% slowdown through pollution of the small L1D. Late ratios at L2 exceed 0.9 for the spatial prefetchers — with so few misses and long compute phases, timeliness hardly matters; even late prefetches arrive during computation. The benchmark's main role in the study is negative control: prefetching has limited impact because locality is already strong, and in these runs the only observed slowdown comes from aggressive prefetching into the small L1D.

Merge sort is the one benchmark AMPM wins in all four cases (1.16–1.20), although the margins over Next-line and Stride are small ($\leq$0.02). Merging traverses several sequential runs simultaneously, which is precisely the multi-stream spatial pattern AMPM's access maps are built to capture. It's best configurations combine moderate accuracy (0.13–0.15) with the highest L1D coverage (0.26–0.29) at very low unused ratios (<0.01). The gains are bounded near 1.2 because merge sort's baseline load-to-use latency is only 6.7 cycles, where most of its time is comparison and copy work. IMP is competitive only after tuning (+4.5%) and only because its stream detector covers the same sequential runs.

Quick sort shows the tightest packing of all benchmarks: at L1D/DDR4 1x, the top three families land at 1.183, 1.183, and 1.178. Partition scans are linear sweeps, so every prefetcher covers them (coverage 0.85–0.97), L1D demand misses are almost eliminated, and the remaining $\sim$1.18 speedup limit is likely due to non-memory bottlenecks such as data-dependent branches and dependencies. As discussed in Section 4.6, IMP at L2 achieves the best prefetch-quality numbers in the study here and still finishes last. This is the cleanest demonstration that prefetch quality metrics matter only in proportion to the memory-boundedness of the workload.

BFS was expected to favor IMP because of its indirect visited-array accesses, but the default Stride prefetcher wins all four cases (1.29–1.35) and IMP is consistently last or second-to-last at L2 (1.09 at L2/DDR4 1x). Two effects explain this inversion. First, the CSR edge-array scan dominates the miss stream, and BFS's traversal order makes the "indirect" accesses hard for IMP to learn: its detected patterns yield only 0.06–0.18 accuracy, no better than the spatial prefetchers. Second, wasted traffic is expensive here: BFS is the irregular benchmark with the highest DRAM pressure, and Next-line/IMP/AMPM discard 40–51%

of their prefetches, while Stride wastes only 7–11% (Figure 2c). AMPM comes closest to Stride (within 0.01–0.02 in every case) by exploiting the same streams with more machinery. The lesson mirrors the conclusion of Section 4.7: algorithmic irregularity does not imply that an indirect prefetcher wins — the data layout determines the profitable pattern.

5 Conclusion

This project built a complete gem5-based evaluation pipeline comparing AMPM and IMP against Next-line and Stride prefetchers on an out-of-order Armv8 model, across six benchmarks, L1D and L2 placement, two DDR4 memory systems, and more than 3,000 simulations covering both default and systematically tuned configurations. The main findings are:

The dominant access pattern is the strongest predictor of which prefetcher wins in this benchmark set. IMP wins only, but decisively, on SpMV (up to 1.78×), the one benchmark with a learnable indirect pattern. AMPM wins the multi-stream merge sort. Simple Stride wins BFS in every case, and simple spatial prefetchers win the pure stream. Averaged over all 24 cases, best-tuned Stride (geometric mean 1.27) matches or beats AMPM (1.25), IMP (1.24), and Next-line (1.24).

Algorithmic irregularity does not necessarily imply indirect prefetching wins. BFS is the clearest example of this, as its compact adjacency-array layout turns most misses into sequential scans, Stride's selectivity (7–11% unused prefetches versus 40–51% for the others) preserves scarce bandwidth, and IMP finishes last despite BFS being a "graph workload." The data structure, not the algorithm, determines the profitable pattern.

Placement and bandwidth interact with the benchmark, not the prefetcher. Latency-sensitive irregular codes (BFS, SpMV, the sorts) prefer L1D placement for timeliness; streaming and reuse-heavy codes (Triad, matmult) prefer L2 for capacity and request concurrency, with Triad gaining up to 1.34× additional benefit when memory bandwidth is doubled. Conversely, on latency-bound SpMV, doubling bandwidth slightly *reduces* the relative value of prefetching.

In this benchmark set, the simple prefetcher defaults are more robust, while the complex prefetchers benefit more from tuning. Tuning adds up to 18% on the memory-bound benchmarks, and IMP both shows the largest sensitivity to tuning in these experiments (its multi-parameter configurations consistently beat single-parameter ones) and is the only family whose default causes a slowdown. With tuning, no prefetcher loses to the no-prefetch baseline in any case.

Speedup can be explained only by combining metrics. The study contains configurations with near-perfect accuracy and coverage but the worst speedup of their case (IMP on quick sort at L2), and configurations with near-zero measured accuracy but 1.2–1.5× speedups (all families on Triad at L1D, where in-flight prefetches overlap latency without registering as useful). Accuracy, coverage, unused and late ratios, miss rates, and DRAM pressure metrics were all necessary to explain why each prefetcher wins or loses; no single metric, including speedup itself, is sufficient to explain all cases.

Overall, the results support a pragmatic conclusion for HPC-style workloads: a well-tuned spatial prefetcher is a very strong default, while specialized prefetchers such as IMP are most valuable when their target pattern clearly dominates the miss stream.

References

[1] S. Mittal, "A survey of recent prefetching techniques for processor caches," *ACM Comput. Surv.*, vol. 49, pp. 35:1–35:35, Aug. 2016. https://doi.org/10.1145/2907071.

[2] B. Falsafi and T. F. Wenisch, *A Primer on Hardware Prefetching*. Synthesis Lectures on Computer Architecture, Morgan & Claypool Publishers, 2014. https://ieeexplore.ieee.org/servlet/opac?bknumber=6828870.

[3] S. Srinath, O. Mutlu, H. Kim, and Y. N. Patt, "Feedback directed prefetching: Improving the performance and bandwidth-efficiency of hardware prefetchers," in *2007 IEEE 13th International Symposium on High Performance Computer Architecture*, pp. 63–74, 2007. https://doi.org/10.1109/HPCA.2007.346185.

[4] N. Binkert, B. Beckmann, G. Black, S. K. Reinhardt, A. Saidi, A. Basu, J. Hestness, D. R. Hower, T. Krishna, S. Sardashti, R. Sen, K. Sewell, M. Shoaib, N. Vaish, M. D. Hill, and D. A. Wood, "The gem5 simulator," *SIGARCH Comput. Archit. News*, vol. 39, pp. 1–7, Aug. 2011. https://doi.org/10.1145/2024716.2024718.

[5] Y. Ishii, M. Inaba, and K. Hiraki, "Access map pattern matching for high performance data cache prefetch," *Journal of Instruction-Level Parallelism*, vol. 13, 2011. https://jilp.org/vol13/v13paper3.pdf.

[6] X. Yu, C. J. Hughes, N. Satish, and S. Devadas, "IMP: indirect memory prefetcher," in *Proceedings of the 48th International Symposium on Microarchitecture*, MICRO-48, (New York, NY, USA), pp. 178–190, Association for Computing Machinery, 2015. https://doi.org/10.1145/2830772.2830807.

[7] A. J. Smith, "Cache memories," *ACM Comput. Surv.*, vol. 14, pp. 473–530, Sept. 1982. https://doi.org/10.1145/356887.356892.

[8] T.-F. Chen and J.-L. Baer, "Effective hardware-based data prefetching for high-performance processors," *IEEE Transactions on Computers*, vol. 44, no. 5, pp. 609–623, 1995. https://doi.org/10.1109/12.381947.

[9] B. Saglam, N. Ho, C. Falquez, A. Portero, F. Schätzle, E. Suarez, and D. Pleiter, "Data prefetching on processors with heterogeneous memory," in *Proceedings of the International Symposium on Memory Systems*, MEMSYS '24, (New York, NY, USA), pp. 45–60, Association for Computing Machinery, 2024. https://doi.org/10.1145/3695794.3695800.

[10] N. Ho, C. Falquez, A. Portero, E. Suarez, and D. Pleiter, "Memory prefetching evaluation of scientific applications on a modern HPC Arm-based processor," *IEEE Access*, vol. 13, pp. 85898–85926, 2025. https://doi.org/10.1109/ACCESS.2025.3569533.

[11] J. Lowe-Power *et al.*, "The gem5 simulator: Version 20.0+," 2020. arXiv:2007.03152. https://arxiv.org/abs/2007.03152.

[12] T. Nowatzki, "CS 251A microbenchmarks." Course microbenchmark collection, UCLA PolyArch group. https://github.com/PolyArch/cs251a-microbench.

[13] J. Lowe-Power, "gem5 course assignments, computer architecture course at UC Davis." https://github.com/jlpteaching/gem5-assignment3-wq25.

[14] J. D. McCalpin, "STREAM: Sustainable memory bandwidth in high performance computers," tech. rep., University of Virginia, Charlottesville, Virginia, 1991-2007. A continually updated technical report. http://www.cs.virginia.edu/stream/.